TL;DR

UTM Architecture & Multi-Agent Reinforcement Learning

The central finding: Current Unmanned Aircraft System Traffic Management (UTM) systems are primarily designed to coordinate, monitor, and deconflict drone traffic using predefined rules and trajectory planning. **Multi-Agent Reinforcement Learning (MARL) could make these systems more adaptive, but it cannot independently guarantee aviation safety.**

1. What Lockheed Martin and RTX are doing

- **Lockheed Martin:** Focuses on vehicle control, autonomous mission execution, sensor integration, and distributed coordination.
- **RTX / Raytheon:** Strong in radar surveillance, air traffic management infrastructure, and airspace situational awareness.
- **Main limitation:** Their complete internal UTM architectures and algorithms are not publicly documented, so specific implementation claims remain uncertain.

2. How UTM works

Sensors + Drone Telemetry + Weather

↓

Airspace State Estimation

↓

Conflict Detection + Trajectory Planning

↓

Safety Verification + Deconfliction

↓

Commands to Drones + Continuous Monitoring

3. Where existing systems struggle

Problem	Why it matters
Dense drone traffic	Traditional planners become computationally expensive
Communication failures	Drones may operate on outdated information
Uncertain trajectories	Wind, delays, and sensor errors affect predictions
Decentralized decisions	Independently planned routes can conflict
Safety guarantees	Good average performance does not establish collision-free operation

4. Where MARL fits

Instead of relying entirely on fixed optimization rules, drones or regional controllers learn cooperative policies.

A simplified objective is:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R_t\right]$$

Subject to safety constraints such as:

$$P(\text{collision}) \leq \epsilon$$

MARL could improve traffic throughput, adapt to unexpected disturbances, and coordinate multiple drones simultaneously.

But the major problem is that MARL policies are difficult to formally verify. A policy that performs well across millions of simulations can still fail in rare, safety-critical situations.

5. Most promising research direction

A **hybrid MARL + formal safety architecture**:

- **MARL:** Learns efficient cooperative trajectories.
- **Model Predictive Control (MPC):** Optimizes trajectories over a moving planning horizon.
- **Control Barrier Functions (CBFs):** Enforce local safety constraints when their assumptions hold.
- **Runtime safety shield:** Rejects unsafe actions and invokes a verified fallback.
- **Formal verification:** Establishes guarantees for specified models and operating conditions.

Research potential

Pure MARL for drone routing (Limited novelty)

Already extensively studied, with substantial challenges in safety certification and generalization.

MARL + safety shielding (Promising)

Stronger research direction, but existing safe-RL literature means novelty must come from a specific technical contribution.

Scalable, uncertainty-aware, formally constrained MARL (High potential)

Particularly interesting for dense traffic, communication loss, and unseen multi-agent configurations.

Bottom line: The opportunity isn't simply making drones smarter with reinforcement learning. It's developing a UTM architecture that combines **learned coordination, scalable trajectory optimization, and defensible safety guarantees**, especially under uncertainty and system failures.

The strongest research question is:

> *Can formally constrained MARL improve airspace throughput and reduce delays compared with conventional UTM planning, while maintaining safety under dense traffic, uncertain trajectories, and communication failures?*

Reading guide and evidence boundaries

Documented capability means a function described in a vendor, agency or research source; vendor descriptions are not independent performance validation. **Reference architecture** and **proposed design** mean an engineering synthesis, not a disclosed vendor implementation. Mathematical examples are explanatory models rather than approved operational thresholds.

The bibliography links the sources used to repair the supplied citation markers. Public standard scopes and catalog entries were reviewed; full paid standards were not audited. Confirm edition, change notices, applicability and means of compliance before using this report in an assurance program. No new simulation, benchmark or flight experiment was performed.

The report preserves the five requested research sections. Diagrams show proposed integration; the comparison matrix is qualitative. The supplied text has been corrected where reward-model terminology, reachability assumptions or attribution could otherwise overstate the evidence.

Contents

Reading guide and evidence boundaries 3

1 Enterprise system decomposition 4

2 Mathematical failure modes, fragility, and edge-case regimes 8

3 MARL in UTM and current organizational objectives 11

4 Safe RL and formal verification integration 15

5 Comparative evaluation and open frontiers 19

Evaluation protocol and unresolved proof obligations 24

References 25

Key notation. N: aircraft count; K: planning intervals; k: local neighbor count; p, v, u: position, velocity and control; h: barrier; V: reachability value; $\mathcal{U}, \mathcal{W}$: admissible control and disturbance sets. UTM: UAS traffic management; USS: UAS service supplier; DAA: detect and avoid; C2: command and control; NMAC: near mid-air collision; LoWC: loss of well clear; MARL: multi-agent reinforcement learning.

A central finding of the public evidence is that **neither Lockheed Martin nor RTX publicly exposes an end-to-end "UTM algorithm stack" at the level needed to truthfully claim that a specific proprietary deployment uses Hybrid A*, kinodynamic RRT*, a particular MILP formulation, or a particular reinforcement-learning policy**. Lockheed publicly documents VCSi as modular vehicle-control software, Hydra Fusion Tools as a geospatial/data-fusion capability, and its Skunk Works "Project Hydra" as an open-systems communications/interoperability experiment; RTX publicly documents Skyler as a non-cooperative surveillance radar and STARS as terminal air-traffic automation. FAA/NASA material, meanwhile, documents the service architecture of UTM itself. The rigorous way to analyze these systems is therefore to separate **documented product functions** from a **reference UTM integration architecture** and from **algorithm families that can implement those functions**. [1,2,3,4,5]

That distinction matters especially for "Hydra." Lockheed's Hydra Fusion Tools product is a real-time geospatial/fusion capability associated with UAS information products, while the Skunk Works Project Hydra demonstration used a U-2-hosted Open Systems Gateway to exchange data among F-22s, F-35s, and ground elements. Project Hydra is strong evidence of Lockheed's distributed interoperability philosophy, but it is **not evidence that "Hydra" is the optimizer inside an operational UTM service**. [2,3]

1 Enterprise system decomposition

A defensible enterprise reference topology. Lockheed's VCSi belongs primarily at the vehicle-command/mission-management edge. Lockheed describes VCSi as modular, extensible vehicle-control software, and has demonstrated it controlling multiple unmanned vehicles; that makes it an appropriate endpoint for mission intent, health, command, handoff, and telemetry, but not by itself a USS or UTM network. [1] RTX's Skyler belongs in the surveillance layer: RTX describes it as a software-defined AESA non-cooperative radar capable of detecting and tracking aircraft and drones without requiring cooperative transponders. STARS, by contrast, is the FAA's terminal automation system sustained at TRACONs and associated towers; it should be understood as an ATM boundary system, not as an ASTM-F3548-native USS. [4,5]

The public UTM architecture provides the missing middle. FAA's UTM Concept of Operations assigns USSs functions such as operation planning, intent sharing, strategic/tactical conflict-management support, conformance monitoring, Remote ID-related services, airspace management, and off-nominal management. Multiple USSs exchange information through a network rather than requiring every operator to interface directly with FAA systems, while supplemental data providers can furnish specialized surveillance, terrain, obstacle, and weather data. [6]

A realistic enterprise topology is therefore:

Proposed reference architecture

Aircraft and local flight control / DAA ↔ vehicle control station
↕ telemetry, health and mission intent
Operator / USS: operation volumes, deconfliction, conformance and constraints
↔ other USSs; ← surveillance, weather and terrain providers
↕ regulated interfaces
FAA / NAS information services and terminal ATM boundary

The important systems-engineering point is that **surveillance, vehicle control, USS coordination, and ATC automation occupy distinct assurance domains**. It would be architecturally hazardous to treat a Remote ID observation, a USS intent reservation, a radar track, and a flight-control state estimate as interchangeable simply because all contain position-like data. FAA's UTM architecture itself separates operators, USS services, supplemental data providers, and FAA interfaces. [6]

Standards mapping. ASTM F3411 and F3548 are primarily interoperability/service-layer standards, whereas the named RTCA documents primarily address airborne or surveillance performance. The ASTM public scopes distinguish Remote ID message exchange from USS interoperability and strategic coordination. [7,8]

Standard family	Architectural placement	Primary role in this reference stack	What it should not be mistaken for
ASTM F3411	Vehicle/Remote-ID service/observer boundary	Remote identification and tracking information exchange; See the ASTM Remote ID scope. [7]	A collision-avoidance controller or guaranteed-separation service
ASTM F3548	USS-to-USS service mesh	USS interoperability, strategic conflict detection/coordination, and discovery/exchange of operation information. [8]	A hard-real-time C2 protocol
RTCA DO-365C	Airborne DAA assurance layer	MOPS for DAA systems. Applicability must be checked against the selected edition and operating class; this is not a generic small-drone UTM specification. [9]	USS interoperability
RTCA DO-381/DO-381A family	Ground-surveillance input to DAA	Performance standardization associated with ground-based surveillance used in DAA architectures. RTCA's current catalog includes the DO-381A revision. [10]	A network-UTM strategic planner
RTCA DO-386 Volumes I/II	Airborne collision-avoidance logic	ACAS Xu MOPS and its associated algorithm-design description; RTCA identifies ACAS Xu as intended for UAS with varying surveillance capabilities. [11]	Small-UAS Remote ID or an enterprise USS

This produces a useful separation of concerns:

$$\underbrace{\text{F3411}}_{\text{identity/track dissemination}} \neq \underbrace{\text{F3548}}_{\text{cooperative intent coordination}} \neq \underbrace{\text{DO-365/381/386}}_{\text{DAA/surveillance/collision-avoidance performance}}.$$

A robust enterprise stack consumes all three kinds of information but maintains **provenance and assurance labels** on each data item.

Strategic deconfliction is fundamentally a four-dimensional set-intersection problem. NASA's UTM demonstrations used discoverable four-dimensional operation volumes and strategic deconfliction/negotiation rather than requiring a monolithic centralized controller. NASA describes strategic deconfliction as the first layer in a conflict-management hierarchy, mostly pre-departure but also usable en route when enough planning horizon exists. [6]

Represent operation i by a time-indexed occupied set

$$\mathcal{V}_i(t) = \left\{ p \in \mathbb{R}^3 : p \in \mathcal{T}_i(t) \oplus \mathcal{E}_i(t) \right\},$$

where $\mathcal{T}_i(t)$ is the nominal trajectory tube and $\mathcal{E}_i(t)$ is an uncertainty/safety envelope. Two reservations conflict if

$$\exists t \in [t_0, t_f] \quad \mathcal{V}_i(t) \cap \mathcal{V}_j(t) \neq \varnothing.$$

This is much more expressive than testing whether two centerlines intersect: $\mathcal{E}_i$ can absorb navigation error, tracking error, aircraft dimensions, wake/wind allowances, communications latency, and required separation.

A discretized reservation implementation introduces cells $c \in \mathcal{C}$ and time bins $k \in \{1, \dots, K\}$:

$$x_{i,c,k} \in \{0, 1\},$$

with an exclusivity or capacity constraint such as

$$\sum_i x_{i,c,k} \leq C_{c,k}.$$

Vehicle connectivity and kinodynamics impose

$$x_{i,c,k} = 1 \Longrightarrow \sum_{c' \in \text{Reach}(c, \Delta t)} x_{i,c',k+1} = 1.$$

The grid is therefore not merely a database reservation; it is a discretized reachable-set graph.

For continuous vehicles, the planning problem is

$$\dot{x} = f(x, u), \qquad x(t) \in \mathcal{X}_{\text{free}}(t), \qquad u(t) \in \mathcal{U},$$

with objective

$$J = \int_{t_0}^{t_f} \left[w_t + w_E P(x, u) + w_\Delta \|u\|^2 + w_r \, \rho_{\text{risk}}(x, t) \right] dt.$$

Kinodynamic randomized planners attack that problem directly in state space under dynamic constraints; Hybrid A *uses motion primitives or continuous-state expansion while retaining A*-like graph search. These are established algorithmic families for dynamically constrained planning, but **there is no public evidence in the sources reviewed that Lockheed VCSi or RTX STARS/Skyler specifically embeds either planner**, so attributing them to those products would be speculation. [12,13]

A MILP-based planner instead selects among route/time alternatives. With candidate trajectory m for aircraft i,

$$y_{im} \in \{0, 1\}, \qquad \sum_m y_{im} = 1,$$

and conflicts between candidate pair $(i, m), (j, n)$ can be prohibited through

$$y_{im} + y_{jn} \leq 1$$

whenever their protected volumes intersect. More detailed formulations use binary variables to encode which side of a separating hyperplane an aircraft passes, sequencing constraints, altitude layers, or time precedence. Mixed-integer formulations of multi-aircraft conflict resolution are well established, with computational burden increasing sharply as the number of coupled conflicts grows. [14]

Priority can be represented either through hard constraints,

$$y_{im} = 1 \quad \forall (i,m) \in \mathcal{P}_{\text{protected}},$$

or large penalties,

$$J = \sum_i c_i^\top y_i + M \sum_{i \in \mathcal{P}} \mathbf{1}[\text{priority trajectory altered}],$$

although a lexicographic optimizer is usually preferable to a numerically dangerous "big-M" surrogate when priority is truly categorical.

Tactical DAA is a different timescale. ACAS Xu addresses larger UAS, ACAS sXu addresses small drones, and ACAS Xr targets rotorcraft and advanced air mobility. The MIT Lincoln Laboratory source describes these variants; their surveillance, dynamics and advisory assumptions differ. This is not evidence that a particular vendor deploys one of them. [15]

The underlying ACAS-X abstraction is naturally written as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

with an offline Bellman recursion

$$Q(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s'),$$

$$V(s) = \max_a Q(s,a)$$

for a reward-maximization convention. A state may encode relative range, bearing, altitude difference, ownship/intruder velocity, relative heading, closure information, previous advisory, and surveillance-quality variables. The expensive stochastic optimization is performed offline; execution reduces to a structured policy evaluation or lookup/interpolation problem. ACAS Xu is consequently an important example of **computationally expensive synthesis followed by bounded online decision logic**, rather than an online MDP solver. Research has investigated neural-network approximation of ACAS Xu tables, using asymmetric training losses and subtables to preserve advisory preferences while reducing storage. Approximation requires separate verification; it is not evidence of deployment in the named vendor products. [15,16]

A basic tactical screening primitive is closest point of approach. For relative position and velocity

$$r = p_j - p_i, \qquad v = v_j - v_i,$$

constant-velocity CPA occurs at

$$t^*_{\text{CPA}} = -\frac{r^\top v}{v^\top v},$$

with implementation horizon

$$t_{\text{CPA}} = \text{clip}\left(t^*_{\text{CPA}}, 0, T_{\text{look}}\right),$$

and

$$d_{\text{CPA}} = \left\| r + v\, t_{\text{CPA}} \right\|.$$

When $\|v\| \approx 0$, t_{CPA} is ill-conditioned and the implementation must fall back to present separation or another encounter metric.

A conceptual well-clear detector has both horizontal and vertical components:

$$C_H = [d_H \leq D_{\text{THR}}] \vee [0 < \tau_H \leq T_{\text{THR}}]\,,$$
$$C_V = [|h| \leq Z_{\text{THR}}] \vee [0 < t_{\text{coalt}} \leq T_{\text{COA}}]\,,$$
$$\text{LossOfWellClear} = C_H \wedge C_V\,.$$

This simplified detector is not the DAIDALUS well-clear predicate: time-to-conflict alone needs a miss-distance gate. For example, modified tau is (D squared minus horizontal range squared) divided by the horizontal position-velocity dot product when closing; the horizontal gate also checks predicted miss distance. [31] Actual standards contain more carefully defined metrics, filtering, hysteresis, surveillance assumptions, and operating conditions; simple t_{CPA} alone is **not** equivalent to certified ACAS/DAA logic. RTCA's separate MOPS for DAA and ACAS Xu are precisely why a naive CPA threshold should not be elevated to the status of an aviation safety standard. [9,11]

2 Mathematical failure modes, fragility, and edge-case regimes

Dense traffic converts seemingly modest pairwise logic into a combinatorial problem. For N aircraft there are

$$M = \binom{N}{2} = \frac{N(N-1)}{2}$$

possible pair interactions. At only $N = 100$,

$$M = 4{,}950.$$

With $K = 60$ planning intervals, a formulation containing one separation predicate per pair and interval already has

$$MK = 297{,}000$$

pair-time conditions before accounting for route alternatives, altitude choices, priority rules, uncertainty modes, or contingency branches.

If each conflict requires a binary disjunction, a centralized MILP can acquire

$$B = \Theta(N^2 K)$$

integer decisions. Branch-and-bound does **not** normally enumerate all 2^B assignments, but 2^B captures why no polynomial worst-case solve-time guarantee should be assumed. Aircraft conflict-resolution research has long used mixed-integer formulations while recognizing their computational burden. [14]

This distinction is important when somebody claims a UTM algorithm is "$\mathcal{O}(N^2)$." Pairwise **conflict detection** can indeed be approximately quadratic under all-to-all checking:

$$T_{\text{detect}} = O(N^2 K).$$

The resulting **integer optimization** is not generally $O(N^2)$; its worst-case combinatorial search is much worse.

Locality changes the problem. Let $G = (V, E)$ be a proximity graph and bound each aircraft's relevant neighborhood by $k \ll N$. Then

$$|E| = O(Nk),$$

so distributed tactical processing can approach linear scaling in fleet size if physical density, sensor horizon, and communication structure keep k bounded. In ultra-dense airspace, however, k itself grows and this advantage collapses.

State-estimation failures are often more dangerous than planner failures because the optimization may remain internally consistent while optimizing the wrong state. A classical linear-Gaussian estimator assumes

$$x_{t+1} = Fx_t + w_t, \qquad z_t = Hx_t + v_t,$$

with independent Gaussian noises and a Gaussian initial state. Under those linear-Gaussian assumptions, the Bayesian posterior remains Gaussian. In an urban multipath/outlier regime, however, a more realistic likelihood may resemble

$$p(z|x) = (1-\epsilon)\mathcal{N}(Hx, R) + \epsilon q(z),$$

where q represents multipath ghosts, clutter, incorrect data associations, spoofed observations, or heavy-tailed errors.

A single Gaussian posterior then compresses potentially multimodal hypotheses into one mean/covariance. That creates a particularly serious UTM pathology:

$$\text{small reported } P \not\Rightarrow \text{ small epistemic error.}$$

The estimator can be *confidently wrong*.

Data association itself becomes combinatorial. With n tracks and m measurements, the number of possible partial one-to-one assignments is

$$A(n,m) = \sum_{k=0}^{\min(n,m)} \binom{n}{k}\binom{m}{k} k!,$$

before gating and probabilistic pruning. Dense clutter and multipath therefore attack both statistical correctness and computation simultaneously.

RTX advertises Skyler's digital beamforming/processing specifically for tracking and clutter reduction, which is operationally valuable, but no radar-processing technique eliminates the fundamental observability problem when two physical hypotheses induce nearly indistinguishable measurements. This is a generic estimation limitation, not a claim of a product-specific defect. [4]

An enterprise fusion system should therefore propagate a **belief or integrity envelope**, not merely $(\hat{x}, \hat{v})$:

$$b_i = (\hat{x}_i, P_i, \text{source provenance}, t_{\text{stamp}}, q_{\text{integrity}}, \mathcal{H}_i)\,,$$

where $\mathcal{H}_i$ may contain competing track hypotheses.

C2 latency has an explicit physical breaking point. Define the end-to-end delay

$$\tau_{\text{e2e}} = \tau_{\text{sense}} + \tau_{\text{estimation}} + \tau_{\text{uplink}} + \tau_{\text{compute}} + \tau_{\text{downlink}} + \tau_{\text{actuator}}.$$

Let t_{loss} be the predicted time until the system crosses the recoverability boundary and t_{maneuver} the time needed for a dynamically feasible avoidance maneuver. The tactical margin is

$$M_t = t_{\text{loss}} - \tau_{\text{e2e}} - t_{\text{maneuver}}.$$

If

$$M_t \le 0,$$

no improvement to the *remote* planner's optimality can repair the architecture: by the time the command can have physical effect, the maneuver is no longer guaranteed to be recoverable.

That is why tactical collision avoidance belongs as close to the aircraft as practical. USS networking remains valuable for larger-horizon coordination, but FAA's UTM concept explicitly describes networked USS information exchange and separately acknowledges preprogrammed/contingency behavior for events such as lost C2. [6]

It is also important to separate **C2 degradation** from **ASTM F3548 synchronization degradation**. F3548-oriented USS interoperability is not the aircraft command link. Stale USS intent instead creates a strategic consistency problem. Define age of information

$$\Delta_i(t) = t - u_i(t),$$

where $u_i(t)$ is the timestamp of the newest accepted intent/state update. Even with zero corrupted packets, deconfliction can be invalid when

$$\Delta_i(t) > \Delta_{\text{max}}$$

for the assumptions used by the strategic planner.

The dangerous case is not always an obvious disconnect. A message may be correctly authenticated, syntactically valid, and semantically obsolete. Enterprise mitigations therefore need monotonically increasing intent versions, bounded time-to-live, clock-quality monitoring, explicit stale-data states, conformance monitoring, and a rule that loss of coordination causes **graceful reduction to local safety behavior**, not optimistic continuation.

Rigid four-dimensional reservations break under disturbance-model mismatch. Consider a translational model

$$\dot{p} = v, \qquad \dot{v} = u + w(p,t),$$

where w is wind-induced acceleration/disturbance. A deterministic planner effectively assumes

$$w \in \mathcal{W}_{\text{model}}.$$

For a chosen tracking policy pi, a robust tube must contain its reachable states under all modeled disturbances (not all arbitrary control inputs):

$$\text{Reach}_{\pi}\left(x_0, \mathcal{W}_{\text{model}}, t\right) \subseteq \mathcal{V}(t).$$

If a building corner, rooftop shear layer, street-canyon jet, or downwash event generates

$$w_{\text{actual}} \notin \mathcal{W}_{\text{model}},$$

the reservation's proof obligation no longer holds.

For a simple double-integrator and an uncompensated constant disturbance of magnitude $\bar{w}$,

$$\|e_p(t)\| \simeq \frac{1}{2}\bar{w}t^2.$$

Thus a small increase in disturbance magnitude or replanning latency produces a quadratic growth in position deviation over the open-loop interval. FAA's UTM architecture anticipates specialized weather as an external data service, but incorporation of weather data is not the same as guaranteeing that the micro-scale field affecting a vehicle is resolved accurately enough for tight urban corridor reservations. [6]

The general robust-planning correction is

$$\mathcal{V}_i(t) = \mathcal{T}_i(t) \oplus \mathcal{R}_i^{w}(t) \oplus \mathcal{R}_i^{\text{nav}}(t) \oplus \mathcal{R}_i^{\text{latency}}(t),$$

where the Minkowski-summed terms are disturbance, navigation, and latency reachable sets. The cost is capacity: every additional uncertainty term makes the protected tube wider.

A deeper failure mode follows: **strategic capacity and robustness are mathematically antagonistic**. Larger tubes improve safety against uncertainty but reduce the number of simultaneous reservations. Smaller tubes increase nominal throughput but transfer risk to tactical DAA.

Several pathological regimes recur across deterministic multi-agent systems:

Regime	Mathematical cause	Observable behavior
Reciprocal avoidance oscillation	Both agents independently solve symmetric optimization problems with the same new information	Left-right-left or altitude oscillations
Deadlock	Each trajectory sees the other's reservation as a hard constraint	Agents stop or repeatedly replan
Priority inversion	Soft penalty for violating high-priority traffic is outweighed by many lower-level costs	Low-priority flow blocks urgent operation
Reservation thrashing	Replanning period is shorter than system settling/propagation time	Continual invalidation of neighbors' plans
Estimator-planner inconsistency	Planner uses covariance or state snapshot older than estimator's current belief	"Safe" plan is safe for a state that no longer exists
Feasible-plan/unsafe-execution gap	Kinodynamic model omits actuator, wind, latency, or tracking limitations	Mathematical path cannot be safely flown
Local optimality trap	Pairwise resolutions do not jointly solve a multi-agent geometry	Avoidance of one intruder creates another conflict

The key architectural lesson is that deterministic planning does not fail merely because its solver is "too slow." It fails through a chain:

$$\text{model error} \rightarrow \text{state error} \rightarrow \text{constraint error} \rightarrow \text{optimization over wrong feasible set.}$$

3 MARL in UTM and current organizational objectives

There is **no FAA-standardized MARL reward function for UTM**. Current NASA work should be described as research, not a certified FAA reward specification. A concrete example is the 2025 NASA tactical-separation framework: Double DQN chooses heading changes in pairwise encounters, with intermediate energy/progress rewards and terminal conflict-timing/extra-energy terms. Its reported preliminary experiment uses 150 encounter geometries and 50 epochs. It does not establish dense-fleet MARL safety. [17]

The natural formal model is a cooperative decentralized partially observable Markov decision process (Dec-POMDP), with a shared team reward. The more general agent-specific reward tuple below is a partially observable stochastic game; it reduces to a Dec-POMDP when all agents share the reward:

$$\mathcal{M} = \langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, P, \{\mathcal{O}_i\}, O, \{R_i\}, \gamma \rangle \,,$$

where $\mathcal{I} = \{1, \ldots, N\}$ is the vehicle set; $s_t \in \mathcal{S}$ contains the joint physical/traffic state; $a_i \in \mathcal{A}_i$ is aircraft i's action; $P(s'|s, \mathbf{a})$ describes dynamics and disturbances; and each aircraft receives only

$$o_i \sim O_i(\cdot|s),$$

not the full joint state.

A realistic observation vector is

$$o_i = [x_i, \hat{\mathcal{N}}_i, \Sigma_i, \mathcal{C}_i, w_i, \mathrm{SoC}_i, q_i^{\mathrm{C2}}, \text{mission phase}],$$

where $\hat{\mathcal{N}}_i$ is a set/graph of local intruder tracks and $\mathcal{C}_i$ includes corridor/geofence constraints.

A decentralized policy is

$$a_i \sim \pi_{\theta_i}(a_i|o_i, h_i),$$

with recurrent memory h_i if necessary. Centralized-training/decentralized-execution architectures may train critics using more joint information while keeping flight-time actors local. [32]

A proposed composite reward, illustrating common design choices rather than reproducing an institutional specification, is

$$\begin{aligned} r_i^t = & -C_{\mathrm{NMAC}}\mathbf{1}_{\mathrm{NMAC}} - C_{\mathrm{WC}}\mathbf{1}_{\mathrm{LoWC}} \\ & - \lambda_\tau \frac{\mathbf{1}_{\text{closing}}}{\epsilon + \tilde{t}_{\mathrm{CPA}}} + \lambda_p \left(d_{g,i}^t - d_{g,i}^{t+1}\right) \\ & - \lambda_e \|e_{\text{track},i}\|^2 - \lambda_u \|u_i\|^2 \\ & - \lambda_{\Delta u} \|u_i^t - u_i^{t-1}\|^2 - \lambda_j \|j_i\|^2 \\ & - \lambda_E \Delta E_i - \lambda_c C_i^{\text{coordination}}. \end{aligned}$$

This is an illustrative synthesis. The actual NASA example uses intermediate reward w1 times incremental energy plus w2 times distance progress, and terminal reward 0.75 plus w3 times a late-resolution indicator plus w4 times extra energy. Its weights are (-0.002, 0.001, -0.5, -0.001); these depend on that simulator and are not transferable safety parameters. [17]

The terms have distinct meanings.

Safety uses a severe event penalty,

$$r_{\mathrm{NMAC}} = -C_{\mathrm{NMAC}}\mathbf{1}[d_H < D_{\mathrm{NMAC}} \wedge |d_V| < H_{\mathrm{NMAC}}],$$

plus a larger-horizon well-clear term. A smooth shaping signal can depend on risk,

$$r_{\mathrm{risk}} = -\lambda_\tau \frac{\sigma(-\dot{d})}{\epsilon + \max(0, t_{\mathrm{CPA}})},$$

but clipping is essential because a literal $1/t_{\mathrm{CPA}}$ becomes singular.

Progress is better represented by **distance closure** than a one-time waypoint reward:

$$r_{\text{prog}} = \lambda_p \left(\|p_i^t - p_g\| - \|p_i^{t+1} - p_g\| \right).$$

Cross-track fidelity is

$$r_{\text{track}} = -\lambda_e e_\perp^2.$$

Control effort can distinguish total actuation, slew, and jerk:

$$r_{\text{ctrl}} = -\lambda_u \|u_t\|^2 - \lambda_{\Delta u} \|u_t - u_{t-1}\|^2 - \lambda_j \left\| \frac{a_t - a_{t-1}}{\Delta t} \right\|^2.$$

An electric-UAS energy term can use charge depletion directly:

$$r_E = -\lambda_E \left(\text{SoC}_t - \text{SoC}_{t+1} \right).$$

For cooperation,

$$r_i^{\text{MARL}} = \alpha r_i^{\text{local}} + (1 - \alpha) r^{\text{team}},$$

but this merely moves the reward-design problem into the choice of α.

The core defect of scalar reward engineering is that safety becomes a commodity that can be traded against progress. Suppose moving one step gives progress reward $g > 0$ and produces collision probability p with penalty C. A one-step expected comparison is approximately

$$\mathbb{E}[r_{\text{move}}] = g - pC.$$

The agent prefers to move only when

$$g > pC.$$

If C is increased to make safety "very important," sufficiently uncertain environments satisfy

$$pC > g,$$

and the optimal learned response can become **freezing**. If C is reduced, the learner may rationally accept safety risk for throughput.

No weight simultaneously means

"never violate safety"

and

"trade safety continuously against all other terms."

Those are different mathematical specifications.

The discount factor introduces another pathology:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

With $\gamma < 1$, a catastrophic penalty moved farther into the future is discounted. Unless the MDP terminates, constraints are shaped carefully, or safety is enforced separately, the learner may prefer policies that **delay** failure.

Reward overlap creates additional ambiguity. Avoiding all traffic improves safety but destroys progress. Aggressive acceleration can improve both time-to-goal and sometimes separation. Energy conservation can oppose evasive maneuvering. Large waypoint rewards encourage cutting corners. Penalties for deviation may oppose DAA advisories. A scalar objective

$$R = \sum_k w_k r_k$$

hides all of these conflicts inside hand-tuned w_k.

The tuning is also context dependent. Ideally,

$$w_k = w_k(c_t),$$

where

$$c_t = [\text{density, corridor width, weather, visibility, battery, priority, flight phase}].$$

A weight set appropriate for open rural airspace is not obviously appropriate for a narrow urban canyon or an emergency medical mission.

Inverse reinforcement learning is a plausible way to learn the soft part of this objective. [26] Let an expert deterministic solver, validated pilot, ACAS-like policy, or high-fidelity optimization procedure produce trajectories

$$\mathcal{D} = \{\tau_1, \ldots, \tau_M\}.$$

Define feature totals

$$\Phi(\tau) = \sum_t \phi(s_t, a_t).$$

A maximum-entropy-style reward model can assign

$$P_w(\tau|c) = \frac{\exp\left(w(c)^\top \Phi(\tau)\right)}{Z(w,c)},$$

and learn

$$w^* = \arg\max_w \sum_{\tau \in \mathcal{D}} \log P_w(\tau|c) - \beta\|w\|^2.$$

Rather than manually deciding whether cross-track error is "3.7 times more important" than battery consumption, demonstrations infer those relative preferences.

For UTM, an especially useful training set would combine:

$$\mathcal{D} = \mathcal{D}_{\text{expert}} \cup \mathcal{D}_{\text{MILP}} \cup \mathcal{D}_{\text{DAA}} \cup \mathcal{D}_{\text{contingency}}.$$

There is nevertheless a fundamental IRL limitation: many reward functions induce the same optimal policy, and expert behavior may contain conservatism, bias, or operational artifacts. Therefore **IRL should infer efficiency/preferences, not define the hard safety envelope**.

Reward machines are a stronger mechanism for contextuality than continuously retuning scalar weights. A reward machine is a finite-state structure exposing temporal reward logic; the research literature explicitly uses finite-state reward machines to represent temporally extended objectives and exploit that structure during RL. [18]

Let

$$q_t \in Q$$

represent a mission/reward phase, with transition

$$q_{t+1} = \delta\left(q_t, L(s_t, a_t, s_{t+1})\right)$$

and

$$r_t = R_{q_t}(s_t, a_t, s_{t+1}).$$

For UTM, Q might contain

$$Q = \{q_{\text{launch}}, q_{\text{cruise}}, q_{\text{merge}}, q_{\text{conflict}}, q_{\text{contingency}}, q_{\text{landing}}\}.$$

A merge phase can heavily reward schedule/conformance; the conflict state can suppress progress shaping and emphasize resolution quality; the recovery state can reward rejoining a cleared route. Reward machines thereby make the **temporal semantics explicit**, rather than expecting a neural policy to infer them from a single static scalar function. [18]

Safety should still remain outside that automaton as a constraint.

A rulebook gives an explicit priority hierarchy. Rulebooks allow partial orders; the total order below is a proposed illustration requiring operational review, not an approved aviation priority scheme. [27] Define violation metrics

$$\begin{aligned} v_1(\tau) &= \text{collision/NMAC severity}, \\ v_2(\tau) &= \text{well-clear/compliance violation}, \\ v_3(\tau) &= \text{geofence/airspace violation}, \\ v_4(\tau) &= \text{mission delay}, \\ v_5(\tau) &= \text{energy/control cost}. \end{aligned}$$

Then impose

$$v_1 \succ v_2 \succ v_3 \succ v_4 \succ v_5$$

or a partial order where appropriate. Policies are compared lexicographically, not through

$$\sum_k w_k v_k.$$

This eliminates the nonsensical possibility that enough milliseconds of schedule improvement compensate for a collision.

Automated scenario generation should then **falsify the rulebook/reward pair**, rather than merely estimate average reward. Parameterize a scenario by

$$\theta = [p_{\text{intruder}}, v_{\text{intruder}}, \tau_{\text{C2}}, p_{\text{loss}}, w_{\text{gust}}, \epsilon_{\text{radar}}, \text{density}, \ldots]$$

and search

$$\theta^* = \arg\min_{\theta} \rho\left(\varphi_{\text{safety}}, \tau_\pi(\theta)\right),$$

where ρ is a robustness score for a formal safety specification φ_{safety}. A negative value is a counterexample.

This changes validation from

“mean episodic reward is high”

to

“find the smallest perturbation that breaks a safety requirement.”

That is the appropriate mindset for safety-critical MARL.

4 Safe RL and formal verification integration

The claim that "pure RL cannot be certified under DO-178C Level A" is **too categorical**. DO-178C concerns airborne software assurance; the required software level follows system safety assessment, and it does not automatically apply to every ground USS. [29] EASA's Issue 2 AI concept paper addresses learning assurance and human-AI teaming, not blanket approval of autonomous MARL. [30] DO-178C is not a blacklist saying "RL is prohibited." The deeper problem is producing the evidence expected of safety-critical airborne software - requirements traceability, predictable bounded behavior, verification evidence, configuration control, independence, and coverage - when behavior is encoded statistically and perhaps changes after deployment. The FAA still maintains an explicit AI/ML safety-assurance technical discipline and AI safety-assurance roadmap, with its public AI page updated in April 2026, which shows that acceptable assurance methods remain an active regulatory engineering problem rather than a solved "certified deep RL" pathway. [19]

The practical answer is a **runtime-assurance architecture**:

Proposed reference architecture

Observations, tracks and mission state
↓
Learned MARL policy → nominal action
↓
HOCBF projection + HJ recoverability supervisor
↓
Execute admissible action, or switch early to validated backup
↓
Flight controls

The learning policy supplies **performance**. The shield supplies **constraint satisfaction**. The reachability layer supplies **recoverability**. The deterministic backup supplies a **last-resort behavior whose safety argument does not depend on the learned reward**.

Control Barrier Functions provide the mathematically cleanest online projection layer. For

$$\dot{x} = f(x) + g(x)u,$$

define the safe set

$$\mathcal{C} = \{x : h(x) \geq 0\}.$$

A first-order CBF requires the controller to enforce

$$\dot{h}(x, u) + \alpha(h(x)) \geq 0,$$

or

$$L_f h(x) + L_g h(x) u + \alpha(h(x)) \geq 0,$$

with class-$\mathcal{K}$ function α. Under the associated assumptions, this establishes forward invariance of $\mathcal{C}$. CBF-QP methods were developed specifically to combine performance objectives with safety constraints in real-time optimization. [20]

For two acceleration-controlled aircraft, take

$$h_{ij} = \|p_i - p_j\|^2 - D_{\text{safe}}^2.$$

Then

$$\dot{h}_{ij} = 2(p_i - p_j)^\top (v_i - v_j),$$

which contains no acceleration input. Therefore h has relative degree two and a high-order construction is needed.

Define

$$\psi_0 = h,$$
$$\psi_1 = \dot{\psi}_0 + \alpha_1(\psi_0),$$
$$\psi_2 = \dot{\psi}_1 + \alpha_2(\psi_1).$$

Safety is imposed through

$$\psi_2(x, u) \geq 0.$$

Because

$$\ddot{h}_{ij} = 2\|v_i - v_j\|^2 + 2(p_i - p_j)^\top (u_i - u_j),$$

the highest-order condition becomes affine in the acceleration control.

Given an RL action u_{RL}, solve

$$u^* = \arg\min_u \frac{1}{2}(u - u_{\text{RL}})^\top H(u - u_{\text{RL}}) + \rho\|u - u_{t-1}\|^2$$

subject to

$$\psi_{2,ij}(x, u) \geq 0 \qquad \forall j \in \mathcal{N}_i,$$
$$u_{\min} \leq u \leq u_{\max},$$

plus geofence, attitude, climb-rate, and actuator constraints.

The MARL output is therefore a **proposal**, not an authority:

$$u_{\text{MARL}} \quad \longrightarrow \quad \Pi_{\mathcal{U}_{\text{safe}}(x)}(u_{\text{MARL}}).$$

That is a far stronger assurance structure than attempting to encode

$$-C_{\text{collision}}$$

inside the reward and hoping the learned policy generalizes.

Uncooperative intruders require a robust CBF. If their acceleration satisfies

$$u_j \in \mathcal{U}_j,$$

then enforce

$$\min_{u_j \in \mathcal{U}_j} \psi_{2,ij}(x, u_i, u_j) \geq 0.$$

This converts "the intruder probably maintains velocity" into "the ownship remains safe against every modeled intruder action."

Latency should appear in the barrier geometry as well. A rough protected distance is

$$D_{\text{eff}} = D_0 + v_{\text{rel,max}}\tau + \frac{1}{2}a_{\text{rel,max}}\tau^2 + D_{\text{nav}} + D_{\text{model}}.$$

Thus communications and estimation uncertainty physically enlarge the safe set margin instead of being hidden in a reward coefficient.

CBFs are not magic. The QP can become infeasible when actuator limits make the safe set non-control-invariant, when multiple neighbors impose mutually inconsistent constraints, or when state-estimation error invalidates $h(x)$. A production implementation therefore needs feasibility analysis, robust margins, prioritized constraints, and a backup-recoverability layer rather than simply introducing safety slack and allowing the optimizer to pay for violations.

Guarantee conditions. HOCBF invariance requires initial membership in every auxiliary safe set, feasible constraints, appropriate smoothness, bounded uncertainty and a controller that enforces the inequalities. Sampled execution needs inter-sample margins. Time-varying protected distances require their derivatives in the barrier; pairwise feasibility does not prove simultaneous multi-agent feasibility. [20,28]

Hamilton-Jacobi reachability moves the guarantee from instantaneous constraint enforcement to future recoverability. Consider

$$\dot{x} = f(x, u, d),$$

where u is ownship control and d is an adversarial or worst-case disturbance/intruder action. Let the unsafe target be

$$\mathcal{L} = \{x : l(x) \leq 0\}.$$

A backward reachable set can be written

$$\mathcal{B}(T) = \{x_0 : \exists \beta \; \forall u(\cdot) \; \exists t \leq T : x(t) \in \mathcal{L}\}.$$

Here beta is a nonanticipative disturbance strategy, with d = beta[u]; the information pattern and horizon must be fixed. States in $\mathcal{B}(T)$ are those from which an adversarial disturbance can force entry into the target within the horizon under the chosen game convention.

The corresponding HJ/HJI value function $V(x, t)$ satisfies a PDE whose Hamiltonian contains

$$H(x, \nabla V) = \max_{u} \min_{d} \nabla V^{\top} f(x, u, d)$$

for a controller-maximizes-safety/disturbance-minimizes-safety convention. The exact min/max and sign arrangement varies with value-function convention, but the operational interpretation is stable:

$$V(x) > 0 \Rightarrow \text{outside the computed unsafe BRS},$$
$$V(x) = 0 \Rightarrow \text{recoverability boundary}.$$

HJ methods are explicitly being developed as safety mechanisms for reinforcement-learning systems. [21]

The runtime switching law can therefore be

$$u = \begin{cases} u_{\text{MARL/CBF}}, & V(x) > \epsilon, \\ u^{*}_{\text{HJ}}(x), & V(x) \leq \epsilon. \end{cases}$$

Switch before the boundary using a margin that covers sensing and actuation delay, and use hysteresis to prevent chatter. Arbitrary blending need not preserve safety. A finite-horizon BRS needs a terminal invariant set or a renewed recoverability argument for indefinite operation.

This is particularly attractive for an uncooperative intruder because the BRS can be generated under worst-case intruder behavior, rather than assuming cooperative USS intent.

The cost is dimensionality. A grid with m points per state dimension and n-dimensional state requires roughly

$$O(m^n)$$

grid storage/work. Full multi-aircraft HJ reachability therefore suffers the curse of dimensionality very rapidly. Practical architectures use relative pairwise states, decomposition, conservative projections, neural approximations, or online local solves - but replacing an exact value function with an approximation also weakens the formal guarantee unless the approximation error itself is bounded.

Formal neural-network verification addresses a third assurance layer: the learned function itself. For a policy network

$$y = \pi_{\theta}(x),$$

suppose an input set is

$$X = \{x : A_x x \le b_x\}$$

and a desired output property is

$$A_y y \le b_y.$$

Verification asks whether

$$\forall x \in X : A_y \pi_\theta(x) \le b_y.$$

Equivalently, solve the satisfiability problem

$$\exists x \in X : A_y \pi_\theta(x) \not\le b_y.$$

If the negation is proven unsatisfiable, the property holds over the modeled set.

Marabou is an SMT-style formal-analysis framework for neural networks and has been heavily used with ACAS-Xu-type verification problems. [22] ERAN should be described differently: it primarily uses abstract domains, relaxations, and refinement methods to verify neural-network safety properties; calling both tools simply "SMT solvers" would be inaccurate. [23]

Useful UTM properties include

$$x \in X_{\text{headon}} \Rightarrow a \notin A_{\text{toward}},$$
$$x \in X_{\text{imminent}} \Rightarrow \|\pi_\theta(x)\| \ge a_{\min},$$

or robustness:

$$\forall \delta,\ \|\delta\|_\infty \le \epsilon : \text{class}\left(\pi_\theta(x+\delta)\right) = \text{class}\left(\pi_\theta(x)\right).$$

But **network verification is not closed-loop verification**. Proving

$$\pi_\theta(x) \in U_{\text{allowed}}$$

for all $x \in X$ does not by itself prove

$$x_t \notin X_{\text{collision}} \qquad \forall t.$$

Closed-loop safety additionally depends on

$$x_{t+1} = F(x_t, \pi_\theta(o_t), w_t),$$

the estimator,

$$o_t = H(x_t, v_t),$$

sampling, latency, actuator dynamics, and other agents.

The stronger proof target is therefore

$$\forall x_0 \in X_0,\ w \in W,\ v \in V,\ d \in D : \quad x_t \in X_{\text{safe}} \quad \forall t \le T.$$

This is why SMT/abstract-interpretation verification, CBFs, and HJ reachability are complementary rather than interchangeable.

A credible assurance decomposition is:

$$\boxed{\text{verified NN bounds} + \text{runtime CBF} + \text{HJ recoverability} + \text{deterministic backup}}$$

with the learning component prevented from overriding any higher-assurance layer.

5 Comparative evaluation and open frontiers

The decisive comparison is not "AI versus classical algorithms." It is where each paradigm places complexity and where its safety argument resides.

Property	Enterprise classical UTM	End-to-end deep MARL	Safe-shielded hybrid MARL
Strategic planning	Explicit 4-D intents/reservations, negotiation, optimization	Can learn distributed allocation directly	Usually retains explicit strategic UTM and uses MARL tactically
Online tactical decision	DAA rules, lookup policy, optimization, deterministic state machines	Neural inference directly chooses action	Neural nominal action projected/supervised by formal layer
Centralized conflict scaling	Pair checks $O(N^2K)$; MILP worst-case combinatorial/exponential in integer decisions	Not inherently centralized	Avoids global integer solve if safety remains local
Decentralized inference scaling	Approximately $O(Nk)$ with bounded neighborhoods	$O(N)$ total for fixed-size local observations; graph/attention models can become $O(N^2)$	$O(Nk)$ nominal inference plus local shield cost
Latency predictability	Excellent for lookup/rules; variable for online optimization	Usually bounded neural inference once architecture fixed	Neural inference plus QP/reachability check; typically more predictable than large centralized MILP
Response to novel traffic geometry	Limited by encoded models/rules but behavior is interpretable	Potentially strong interpolation; uncertain OOD behavior	Learning handles nominal complexity while shield bounds unsafe actions
Non-cooperative intruder	Strong when backed by independent radar/DAA	Only as good as observations/training distribution	Strongest if shield/reachability explicitly treats intruder as disturbance
Communication dependence	Strategic layer can degrade if coordination stale; local DAA can remain independent	Can be severe for communication-trained MARL	Can fall back from cooperative policy to local shield/backup
Failure transparency	Comparatively high	Low without interpretability/verification	Medium-to-high because intervention logic is explicit
Worst-case safety case	Strong when models and requirements are valid	Weak for unconstrained learned policy	Stronger within formally modeled envelope
Adaptability	Low-to-medium; requires algorithm/configuration updates	High	High nominally, constrained by certified envelope
Reward-design sensitivity	None for rule-based logic; cost weights still matter to optimizers	Very high	Lower, because safety is removed from reward trade-space
Certifiability / assurance tractability	Highest of the three	Lowest for unconstrained adaptive policy	Most plausible route for learning-enabled safety-critical autonomy
Primary catastrophic failure mode	Model/solver/estimation inconsistency	Distribution shift or reward exploitation	Incorrect shield model or false assumption that the shield covers all hazards

The matrix is a qualitative architecture comparison, not measured vendor benchmarking. A hybrid is not inherently safer than a well-designed classical controller: a wrong shield model can invalidate its guarantee. No relative latency or throughput result is asserted here.

The asymptotic comparison deserves precision. A decentralized neural policy

$$a_i = \pi(o_i)$$

with fixed-size o_i has roughly constant inference cost per aircraft with respect to global N, giving

$$T_{\text{fleet}} = O(N).$$

But if

$$o_i = \{x_j : j \neq i\}$$

and a transformer computes all pairwise attention,

$$T = O(N^2)$$

or worse depending on architecture. MARL therefore does **not automatically solve scaling**; it solves it only if the representation exploits locality, sparsity, hierarchy, or bounded neighborhoods.

For a local CBF-QP with k neighbor constraints and a fixed low-dimensional vehicle-control vector, optimization remains small even as the global fleet grows. A centralized CBF-QP containing all vehicle controls, in contrast, can recreate a large coupled optimization problem. The hybrid architecture should therefore mirror the physical interaction graph whenever possible.

The most promising sim-to-real architecture is hierarchical rather than choosing a single simulator. Flightmare was explicitly designed as a flexible quadrotor simulator with decoupled rendering and dynamics, making it suited to high-rate vehicle/perception training and domain randomization. [24] BlueSky was developed by TU Delft as an open air-traffic simulation environment aimed at reproducible ATM/traffic-flow research. [25]

Their roles are complementary:

$$\text{BlueSky} \rightarrow \text{traffic / UTM / encounter generation,}$$

$$\text{Flightmare} \rightarrow \text{vehicle dynamics / perception / local control.}$$

A credible research pipeline is therefore

Proposed reference architecture

Strategic traffic generator → BlueSky / UTM encounter model
↓
Network and C2 emulator → Flightmare-class vehicle simulation
↓
MARL nominal policy → CBF / HJ runtime assurance
↓
Hardware-in-the-loop → instrumented flight
→ calibrated uncertainty, retraining and falsification

The crucial sim-to-real principle is to randomize *measured uncertainty*, not arbitrary uncertainty. Let simulator parameters be

$$\theta \sim p_{\text{sim}}(\theta),$$

while real parameters follow

$$p_{\text{real}}(\theta).$$

The target is not merely to make p_{sim} "wide." It is to drive a divergence such as

$$D\left(p_{\text{real}} \| p_{\text{sim}}\right)$$

down using flight data, especially for wind spectra, motor lag, battery sag, radar false-alarm distributions, localization biases, network age-of-information, and perception confidence.

An excellent stress curriculum should deliberately sweep the boundaries identified earlier:

$$t_{\text{CPA}} \rightarrow 0,$$

$$M_t \rightarrow 0,$$

$$\text{SoC} \rightarrow \text{reserve},$$

$$\Delta_{\text{USS}} \rightarrow \Delta_{\max},$$

$$w \rightarrow \partial \mathcal{W}_{\text{model}},$$

and

$$\text{neighbor count } k \to k_{\max}.$$

The objective is not millions of benign episodes but concentrated exploration of the **thin failure surfaces** separating recoverable from unrecoverable states.

Byzantine and adversarial behavior is arguably the largest unclosed architectural gap in cooperative UTM/MARL. A cooperative architecture implicitly assumes that a neighbor's message

$$m_j = [p_j, v_j, \text{intent}_j, \text{priority}_j]$$

represents reality. A Byzantine participant can instead transmit

$$\tilde{m}_j = m_j + \delta_j$$

with δ_j intentionally chosen to manipulate another vehicle's planner while remaining statistically plausible.

For a safety-critical system, cryptographic authentication solves only

$$\text{"who sent this message?"}$$

not

$$\text{"is the claimed physical state true?"}.$$

Even a correctly signed aircraft can be compromised.

The minimum architecture therefore needs two independent trust channels:

$$\text{cyber trust} = f(\text{authentication}, \text{freshness}, \text{authorization}),$$
$$\text{physical trust} = g(\text{radar}, \text{EO}, \text{multilateration}, \text{kinematic consistency}).$$

A tactical controller should act on a fused integrity value

$$q_j = Q(q_j^{\text{cyber}}, q_j^{\text{physical}}),$$

rather than trusting a cooperative message merely because its signature is valid.

ADS-B spoofing illustrates the threat model: a received cooperative report must not be treated as proof of physical truth. No spoof-detection performance claim is made here.

A robust UTM stack should therefore require

$$\text{cooperative report} \overset{?}{\approx} \text{independent observation}$$

when independent surveillance exists.

For multiple reports $m_1, \ldots, m_n$, a Byzantine-resistant estimator can replace the arithmetic mean with a robust aggregate such as a geometric median:

$$\hat{m} = \arg\min_z \sum_{j=1}^{n} w_j \|z - m_j\|.$$

But robust statistics alone cannot solve coordinated attackers that form a majority, nor can they detect an adversary whose trajectory claim is physically plausible. That requires redundancy in **sensor modality, trust roots, and physical observability**.

MARL introduces an additional attack surface because the communication message may be a learned latent vector rather than a human-interpretable track:

$$m_{ij} = \phi_\theta(o_j).$$

An attacker does not necessarily need to fabricate an aircraft position; it may instead perturb the message in a direction maximizing the victim policy's loss:

$$\delta^* = \arg\max_{\|\delta\| \le \epsilon} J\left(\pi_i(o_i, m_{ij} + \delta)\right).$$

This motivates adversarial training, message-dropout training, robust aggregation, independent local sensing, and a **communication-denied policy**.

The essential runtime condition should be

$$q_{\text{comm}} < q_{\text{min}} \quad \Longrightarrow \quad \pi_{\text{cooperative}} \to \pi_{\text{local-safe}},$$

not

$$q_{\text{comm}} < q_{\text{min}} \quad \Longrightarrow \quad \text{continue using last cooperative state.}$$

The architecture that emerges from the full analysis is consequently not "replace UTM with MARL." It is a hierarchy:

Regulated UTM intent / reservation services
↓
Distributed strategic deterministic optimization
↓
Context-aware MARL for local tactical performance
↓
HOCBF hard-constraint projection
↓
HJ recoverability supervisor
↓
Verified DAA / deterministic emergency fallback
↓
flight-control system

with surveillance integrity, communication trust, conformance monitoring, and formal scenario falsification spanning every layer.

This hybrid architecture also resolves the deepest reward-design problem. **Hard safety constraints need not be traded against performance through an arbitrary reward coefficient.**

Instead of

$$R = w_s R_{\text{safety}} + w_p R_{\text{progress}} + w_e R_{\text{energy}}$$

and asking whether w_s is "large enough," formulate

$$\max_{\pi} \mathbb{E}\left[R_{\text{progress}} + R_{\text{efficiency}} + R_{\text{comfort}}\right]$$

subject to

$$x_t \in \mathcal{C}_{\text{safe}},$$
$$P(\text{NMAC}) \le \epsilon_{\text{allocated}},$$
$$u_t \in \mathcal{U},$$
$$V_{\text{HJ}}(x_t) \ge 0,$$

and applicable airspace/operational constraints. The risk budget epsilon is an allocated design requirement, not a universal certification threshold. Chance constraints and model-conditional invariance are different guarantees.

The learning system then answers the question it is well suited to answer:

"Among actions admissible under the stated model and uncertainty bounds, which performs best?"

rather than the question for which unconstrained deep RL is poorly suited in a safety-critical aviation system:

"How much collision risk is worth one more unit of mission reward?"

That separation - **learning for optimality, formal methods for admissibility, deterministic runtime assurance for recovery, and enterprise UTM for cooperative strategic coordination** - is the most technically defensible path from today's UTM architectures toward scalable multi-agent autonomy.

Evaluation protocol and unresolved proof obligations

A defensible experiment should compare reservation scheduling with local DAA, receding-horizon optimization, an unshielded learned policy in simulation, and the same policy with its shield. Keep sensors, dynamics, traffic demand, compute budgets and contingency rules matched. Include shield-only and no-communication ablations to distinguish learned coordination from the benefit of the safety controller.

Report throughput and delay at a matched safety target, together with completion rate, minimum separation, NMAC and LoWC frequency, energy, tail compute latency, deadline misses, QP infeasibility and backup interventions. Count all launched missions, including aborts and holds, so a policy cannot appear efficient by rejecting difficult traffic. Report multiple seeds and uncertainty intervals; do not interpret mean reward as a safety metric.

Zero observed failures are not proof of zero risk. For n independent, identically distributed encounters with zero failures, a one-sided 95% upper confidence bound is

$$p_{\text{upper}} = 1 - 0.05^{1/n} \simeq 3/n.$$

Correlated encounters, selected stress tests and changed traffic distributions do not satisfy that simple model. Rare-event methods require validated sampling weights. Failure thresholds and the allocation of safety risk must be defined before comparison.

The most consequential open questions are simultaneous shield feasibility, validity of disturbance bounds during urban gusts, safe transitions under stale intent, recoverability under actuator saturation, and whether local coordination remains effective as neighbor count grows. A count such as 100 aircraft per cubic kilometre is a stress scenario, not a universal solver breakdown threshold. Certification claims additionally require the complete system assurance case and operational approval, not only a neural-network proof or an invariance theorem.

References

Public sources accessed for this edition on 18 October 2026. Source numbers replace the opaque citation markers in the supplied text; they are not the original research-session identifiers.

[1] Lockheed Martin. VCSi product description; modular control, supported interfaces and UTM module status. Open source

[2] Lockheed Martin CDL Systems. Hydra Fusion Tools product brochure. Open source

[3] Lockheed Martin. Project Hydra communications demonstration, 3 May 2021. Open source

[4] RTX / Collins Aerospace. Non-Cooperative Surveillance Radar: Skyler capabilities. Open source

[5] FAA. STARS Maintenance Technical Handbook, Generation 4, Chapter 2, 2015. Open source

[6] FAA. UAS Traffic Management Concept of Operations, Version 2.0, 2020. Open source

[7] ASTM. F3411: Remote ID and Tracking, public scope (2019 edition; catalog identifies later revisions). Open source

[8] ASTM. F3548-21: UTM USS Interoperability, public scope and contents. Open source

[9] RTCA. DO-365C Change 1: MOPS for Detect and Avoid Systems, catalog entry. Open source

[10] RTCA. DO-381A catalog entry; Ground-Based Surveillance System standard family. Open source

[11] NASA. Best Practices for Evaluating DAA Systems, NASA/TM-20205011606, section 4.1.5 on DO-386. Open source

[12] Karaman and Frazzoli. Sampling-based Algorithms for Optimal Motion Planning, 2011. Open source

[13] Dolgov et al. Practical Search Techniques in Path Planning for Autonomous Driving, 2008. Open source

[14] Richards. Trajectory Optimization Using Mixed-Integer Linear Programming, MIT thesis, 2002. Open source

[15] MIT Lincoln Laboratory. Award-winning technology prevents aircraft collisions, 2 March 2023. Open source

[16] Julian, Kochenderfer and Owen. Deep Neural Network Compression for Aircraft Collision Avoidance Systems, 2018 preprint. Open source

[17] Yarramreddy et al. Simulation Framework for Tactical Separation Assurance Service using Deep Reinforcement Learning. NASA/TM-20250002761, March 2025. Open source

[18] Toro Icarte et al. Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning. JAIR 73, 2022. Open source

[19] FAA. Technical Discipline: Artificial Intelligence - Machine Learning; roadmap and assurance resources, updated 9 April 2026. Open source

[20] Xiao and Belta. Control Barrier Functions for Systems with High Relative Degree, 2019. Open source

[21] Fisac et al. A General Safety Framework for Learning-Based Control in Uncertain Robotic Systems, 2017 preprint. Open source

[22] Katz et al. The Marabou Framework for Verification and Analysis of Deep Neural Networks. CAV, 2019. Open source

[23] ETH SRI Lab. ERAN: ETH Robustness Analyzer for Neural Networks, project documentation. Open source

[24] Song et al. Flightmare: A Flexible Quadrotor Simulator, 2020/2021. Open source

[25] Hoekstra and Ellerbroek. BlueSky ATC Simulator Project: An Open Data and Open Source Approach, ICRAT 2016. Open source

[26] Ziebart et al. Maximum Entropy Inverse Reinforcement Learning, AAAI 2008. Open source

[27] Censi et al. Liability, Ethics, and Culture-Aware Behavior Specification using Rulebooks, 2019. Open source

[28] Ames et al. Control Barrier Function Based Quadratic Programs for Safety Critical Systems, 2016/2017. Open source

[29] FAA. AC 20-115D: Airborne Software Development Assurance, 2017. Open source

[30] EASA. Artificial Intelligence Concept Paper, Issue 2: Guidance for Level 1 and 2 machine-learning applications, March 2024. Open source

[31] NASA. Sensor Uncertainty Mitigation and Dynamic Well Clear Volumes in DAIDALUS, DASC 2018. Open source

[32] Lowe et al. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments, 2017. Open source